# Лекция 5. B+-деревья и индексы в базах данных

**Длительность:** ~90 минут
**Темы:** B+-деревья, составные индексы, обработка дубликатов и NULL, кластеризация, физическая упаковка страницы, обзор фильтров Блума, skip-списков, trie-деревьев; реализация в PostgreSQL.

---

## 1. Зачем нужны индексы? (5 мин)

Без индексов СУБД вынуждена выполнять последовательное сканирование (Seq Scan) всей таблицы — читать каждую страницу и проверять каждую строку. На миллиардах записей это неприемлемо.

Индекс — вспомогательная структура данных, реплика подмножества столбцов таблицы, организованная для быстрого поиска нужных кортежей. Индекс отвечает на вопрос: «Где находятся строки с заданным значением ключа?»

Требования к индексу: быстрый точечный поиск, диапазонные запросы, эффективная вставка/удаление, минимизация I/O.

Почти все реляционные СУБД (Oracle, MySQL/InnoDB, PostgreSQL, SQLite, SQL Server) используют B+-дерево или его варианты.

> **Дополнение из курса CMU 15-445/645 «Indexes» (Carnegie Mellon University)**
> В лекциях Энди Павло даётся чёткое формальное определение:
> *«Индекс — это реплика подмножества атрибутов таблицы, организованная и/или отсортированная для эффективного доступа к местоположению конкретных кортежей с использованием этих атрибутов».*
> Важно понимать компромисс: больше индексов ускоряет запросы, но они занимают место и требуют обслуживания при обновлениях. Именно СУБД решает, какие индексы использовать для каждого запроса.
> Также в CMU проводят различие между **индексами** и **фильтрами**:
>
> - **Индекс** хранит подмножество атрибутов и позволяет найти местоположение кортежей (B+-дерево).
> - **Фильтр** отвечает на запросы принадлежности множеству (вероятно, есть ли ключ), но не сообщает, где он находится (фильтр Блума).

---

## 2. B+-дерево: определение и структура (15 мин)

### 2.1. Происхождение (кратко)

- 1970 – Bayer & McCreight (Boeing Scientific Research Laboratories): технический отчёт о B-дереве; каноническая публикация «Organization and Maintenance of Large Ordered Indices» вышла в Acta Informatica в 1972.
- 1979 – Douglas Comer, «The Ubiquitous B-Tree» (ACM Computing Surveys): закрепил B+-дерево (значения только в листьях, связный список листьев) как универсальный индекс.

- 1981 – Philip Lehman (CMU) и S. Bing Yao (Purdue), «Efficient Locking for Concurrent Operations on B-Trees»: **B-link tree**. Указатели на соседние листья существовали в B+-дереве и раньше; вклад Lehman–Yao — link-указатели на правого соседа на **всех уровнях** дерева плюс **high key** на каждой странице. Это позволяет обнаружить конкурентный сплит и «догнать» ключ вправо: писатель держит максимум три блокировки одновременно, читатель — ни одной. Именно этот алгоритм лежит в основе nbtree в PostgreSQL. Оба элемента — high key и rightlink — мы детально разберём в разделе 5, когда дойдём до физической упаковки страницы.

> **Дополнение из CMU 15-445/645 «B+Tree»**
> Существует конкретная структура данных, называемая **B-деревом**, но люди также используют этот термин для общего обозначения класса структур. Основное различие между оригинальным B-деревом и B+-деревом заключается в том, что B+-дерево **хранит значения только в листовых узлах**. Современные реализации B+-деревьев комбинируют возможности из других вариантов, таких как указатели на соседние узлы из **B-link дерева**.

## 2.2. Строгое определение

Зафиксируем конвенцию: **порядок m — максимальное число потомков узла** (в литературе порядок иногда определяют как максимальное число ключей — тогда все формулы сдвигаются на единицу; важно не смешивать две конвенции).

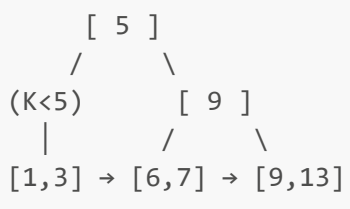
B+-дерево — m-арное сбалансированное дерево поиска со свойствами:

- Идеальная сбалансированность: расстояние от корня до любого листа одинаково.
- Каждый внутренний узел (кроме корня) имеет от ⌈m/2⌉ до m потомков, то есть **от ⌈m/2⌉ − 1 до m − 1 ключей**.
- Узел с *k* ключами имеет *k+1* указателей на потомков.
- Все данные хранятся только в листьях. Внутренние узлы содержат направляющие ключи (guideposts) и указатели на дочерние страницы.

Узел B+-дерева — страница фиксированного размера (в PostgreSQL — 8 КБ).
Листовой узел: пары (key, value). Value — либо физический идентификатор строки (TID, tuple identifier — пара «номер страницы, номер слота»; к роли TID мы вернёмся в разделе 2.4), либо сами данные (кластеризованный индекс).
Внутренний узел: [$ptr_0$, $key_1$, $ptr_1$, $key_2$, $ptr_2$, …, $key_n$, $ptr_n$]. Ключи $key_i$ — направляющие: все ключи в поддереве $ptr_i$ меньше $key_{i+1}$ (для возрастающего порядка). Направляющий ключ может отсутствовать в листьях — это лишь разделитель.

Листовые узлы связаны sibling pointers для диапазонных сканирований без возврата к родителям (в PostgreSQL — двусвязный список: btpo_prev и btpo_next).

Пример дерева (m=3):



Корень с ключом 5: левый потомок — все ключи < 5, правый — внутренний узел [9]: его левый потомок — ключи ≥ 5 и < 9, правый — ≥ 9. Листья связаны указателями слева направо.

> **Дополнение из CMU 15-445/645 (формальные свойства)**
> B+-дерево — это **m-арное поисковое дерево** со свойствами:
>
> - идеально сбалансировано (все листья на одной глубине);
> - каждый внутренний узел, кроме корня, заполнен как минимум наполовину ($\lceil m/2 \rceil - 1 \leq$ ключей $\leq m - 1$);
> - каждый внутренний узел с $k$ ключами имеет $k + 1$ непустых потомков.
>
> **Дополнение из Stanford CS245 «B+tree rules»**
> Правила B+-дерева порядка n (в конвенции «n — число ключей»):
>
> 1. Все листья находятся на одном самом нижнем уровне (сбалансированное дерево).
> 2. Указатели в листьях указывают на записи, за исключением «указателя последовательности» (sibling pointer).
>
> **О структуре узлов (CMU 15-445/645)**
> Каждый узел содержит массив пар **ключ/значение**, почти всегда отсортированный по ключам. Для внутренних узлов значения — указатели на другие узлы, а ключи являются **направляющими столбами (guideposts)**. Они направляют обход, но не обязательно представляют ключи, существующие в листьях. Это означает, что ключ во внутреннем узле (как разделитель) может отсутствовать в листовых узлах.

## 2.3. Составные индексы и префиксный поиск

Если индекс построен по нескольким столбцам, например, (a, b DESC, c NULLS FIRST), то ключ представляет собой кортеж. B+-дерево позволяет искать по префиксу ключа.
Для поиска `WHERE a = x AND b = y` используется полный ключ. Для `WHERE a = x` — поиск по первому атрибуту с последующим последовательным сканированием листьев, пока `a = x`. Это **prefix search**.
Hash-индекс такого не умеет — ему нужны все атрибуты целиком.
Порядок столбцов в составном индексе критичен: для запроса `WHERE b = y` без `a` индекс (a,b) неэффективен.

## 2.4. Обработка дубликатов и NULL

**Дубликаты ключей:**

- **Append Record ID** — к ключу добавляется уникальный TID, делая все ключи уникальными. PostgreSQL делает именно так для неуникальных индексов (heap TID — неявный последний столбец любого nbtree-ключа).
- **Overflow pages** — лист содержит цепочку переполнения. Сложнее в обслуживании, почти не применяется.

**NULL-значения:**
При создании индекса можно указать `NULLS FIRST` или `NULLS LAST`. По умолчанию в PostgreSQL `NULLS LAST` (при ASC; при DESC — наоборот). NULL-ключи размещаются в начале или конце листового списка соответственно.

---

# 3. Операции над B+-деревом (20 мин)

Структура дерева известна — посмотрим, как она работает в динамике. Все три операции начинаются одинаково: спуском от корня к листу. Различия начинаются дальше: при вставке и удалении страницы приходится менять, и именно здесь проявляются ключевые свойства B+-дерева — перестройка всегда локальна, а высота дерева меняется только через корень.

## 3.1. Поиск

1. Начать с корня.
2. На внутреннем узле бинарным поиском найти интервал, содержащий искомый ключ K, перейти по соответствующему указателю.
3. В листе — бинарный поиск точного ключа.

| Случай | Чтений страниц | Комментарий |
|---|---|---|
| Лучший | O(log_m N) | Глубина дерева одинакова для любого ключа |
| Средний | O(log_m N) | Дерево всегда идеально сбалансировано |
| Худший | O(log_m N) | Различается лишь работа внутри страницы: O(log m) бинарным поиском |

Пример (дерево из раздела 2.2): найти 7. Корень: [5]. 7 ≥ 5 → правый указатель. Внутренний узел: [9]. 7 < 9 → левый указатель. Лист: [6,7] → успех.

> **Дополнение из Stanford CS346 «Search Algorithms»**
> Алгоритм поиска: начиная с корня, на каждом внутреннем узле выполняется поиск ключа v среди направляющих ключей для определения правильного дочернего указателя. Процесс продолжается до достижения листового узла, где выполняется поиск записи с ключом v. Если запись найдена, возвращается RID.

## 3.2. Вставка и расщепление

1. Найти лист L.
2. Вставить запись, сохраняя сортировку.
3. Если L переполнен (> m–1 ключей):
    - Создать новый лист L2.
    - Равномерно распределить записи между L и L2.
    - **Скопировать** разделяющий ключ (первый ключ L2) в родительский узел, добавив указатель на L2 (**copy up** — ключ остаётся и в листе).
    - Если родитель переполнен — расщепить его, но средний ключ **перемещается** наверх (**push up** — из узла удаляется).
4. Расщепление может дойти до корня; при расщеплении корня создаётся новый корень, дерево растёт в высоту.

**Пример 1 — сплит листа без каскада** (m=3, max 2 ключа на узел).
Исходное дерево:

```
      [10]
     /    \
[5,7]  [12,15]
```

Вставляем 8. Лист [5,7] → [5,7,8] переполнен. Делим: [5] и [7,8], копируем разделитель 7 наверх. Корень становится [7, 10] — это ровно 2 ключа, переполнения нет. Итог:

```
        [7 | 10]
       /    |    \
    [5]   [7,8]  [12,15]
```

**Пример 2 — каскадный сплит до корня.**
В то же дерево вставляем 17. Лист [12,15] → [12,15,17] переполнен. Делим: [12] и [15,17], копируем 15 наверх → корень [7,10,15] переполнен (3 ключа > 2). Расщепляем корень как внутренний узел: средний ключ 10 **поднимается** (push up) в новый корень:

```
          [10]
         /    \
      [7]      [15]
     /  \      /  \
 [5] [7,8] [12] [15,17]
```

Высота выросла на 1. Обратите внимание на различие: из листа разделитель копируется (15 осталось в листе [15,17]), из внутреннего узла — перемещается (10 удалено из дочерних узлов).

| Случай | Стоимость | Комментарий |
|---|---|---|
| Лучший | O(log_m N) | В листе есть место — спуск и запись |
| Средний | O(log_m N) | Сплиты амортизированно редки |
| Худший | O(log_m N) чтений + O(log_m N) сплитов | Каскад до корня, создание нового корня |

> **Дополнение из CMU 15-445/645 «Insert»**
> При вставке важно различать **copy up** для листьев (копируем разделяющий ключ в родителя) и **push up** для внутренних узлов (поднимаем средний ключ, удаляя его из текущего узла). Это критичное различие, редко объясняемое в учебниках.

### 3.3. Удаление и слияние

1. Найти лист L и удалить запись.
2. Если L стал менее чем наполовину заполнен:
    - **Заимствование**: если соседний узел может отдать одну запись без падения ниже половины, перераспределить записи и обновить разделяющий ключ в родителе.

    - Если заимствование невозможно — **слияние**: объединить L с соседом, удалив разделяющий ключ и указатель в родителе.
3. Каскадное слияние может дойти до корня; если корень потерял все ключи, он удаляется, высота уменьшается.

| Случай | Стоимость | Комментарий |
|---|---|---|
| Лучший | O(log_m N) | Узел остаётся достаточно заполненным |
| Средний | O(log_m N) | Заимствования/слияния амортизированно редки |
| Худший | O(log_m N) чтений + O(log_m N) слияний | Каскад до корня, высота уменьшается |

**Как это на практике в PostgreSQL (nbtree).** Учебный алгоритм заимствования и слияния полупустых страниц в PostgreSQL **не реализован**: объединяться могут только **полностью пустые** страницы. До PostgreSQL 14 пустые страницы лишь помечались и переиспользовались, а вернуть место операционной системе мог только REINDEX; начиная с PostgreSQL 14 VACUUM автоматически удаляет полностью пустые индексные страницы. Обратите внимание: дерево при этом **остаётся строго сбалансированным по высоте** — деградация выражается в *разреженности* страниц (низкой заполненности), а не в разбалансировке.

> **Дополнение из CMU 15-445/645 «Delete»**
> Алгоритм удаления требует попытки перераспределения (заимствования у соседа) перед слиянием. Если слияние произошло, необходимо удалить запись из родителя, указывающую на один из узлов.
>
> **Дополнение из Stanford CS346 «Deletion Complexity»**
> Полностью корректное удаление удивительно сложно. Поэтому многие системы используют **ленивое удаление (lazy deletion)** или **надгробные плиты (tombstones)**, чтобы избежать немедленных слияний. Это объясняет практику PostgreSQL.

---

## 4. Проектные решения и оптимизации (15 мин)

Мы разобрали канонические алгоритмы — в том виде, в каком их дают учебники. Реальные движки от них отступают: разработчикам приходится выбирать размер страницы, обрабатывать ключи переменной длины и решать, что делать с дубликатами. Эти проектные решения — не детали реализации, а ручки настройки, определяющие поведение индекса под нагрузкой.

### 4.1. Размер узла и внутриузловой поиск

- Размер страницы: 8 КБ (PostgreSQL), подбирается под носитель (HDD — крупнее, SSD — 8–16 КБ, in-memory — мельче).
- Поиск внутри страницы: стандарт — бинарный поиск (ключи отсортированы). Реже — линейный (векторизованный) или интерполяционный.

### 4.2. Ключи переменной длины

Применяется карта смещений (Indirection): ключи заменяются ссылками на словарь внутри страницы или в отдельной структуре. PostgreSQL использует этот принцип для работы с кортежами.

### 4.3. Кластеризованные и некластеризованные индексы

- **Кластеризованный**: строки таблицы физически отсортированы по ключу индекса (обычно PRIMARY KEY). Листья содержат сами строки. MySQL/InnoDB, Oracle IOT.
- **Некластеризованный**: листья хранят только ссылки (TID). Таблица — куча. PostgreSQL.

При выборке многих строк по некластеризованному индексу читать страницы кучи в порядке обхода индекса — значит генерировать случайный I/O. Поэтому сначала собирают все TID, упорядочивают их по физическому расположению, и только затем читают страницы таблицы последовательно. В курсе CMU этот приём называется **Index Scan Page Sorting**; в PostgreSQL он реализован как связка **Bitmap Index Scan + Bitmap Heap Scan**: TID накапливаются в битовой карте страниц (при нехватке `work_mem` карта деградирует в lossy-режим, работая на уровне страниц вместо точных кортежей), после чего куча читается в физическом порядке страниц.

> **Дополнение из Stanford CS346 «Primary vs Secondary Index»**
>
> - **Первичный индекс** – отношение физически отсортировано по индексированному атрибуту (⇒ кластеризующий).
> - **Вторичный индекс** – отношение НЕ отсортировано по индексированному атрибуту.
> - **Кластеризующий индекс** – кортежи для каждого значения индекса находятся вместе на страницах, но не обязательно отсортированы.
> - **Плотный индекс** – каждый RID в отношении появляется в индексе. **Разреженный** – некоторые RID не появляются. Разреженный ⇒ первичный или кластеризующий, иначе индекс неприменим.
>
> **Дополнение из Stanford CS245 «Sparse vs. Dense Tradeoff»**
> Разреженный занимает меньше места, можно держать больше в памяти, лучше для вставок. Плотный необходим для вторичных индексов и позволяет определить существование записи без обращения к файлу.

### 4.4. Оптимизации B+-дерева

- **Префиксное сжатие (Prefix Compression)**: общий префикс соседних ключей хранится один раз. Применяется во многих системах (например, в InnoDB — для префиксов колонок и компрессии страниц), но в nbtree PostgreSQL **не реализовано** — существуют лишь исследовательские предложения.
- **Суффиксное усечение (Suffix Truncation)**: во внутренних узлах разделяющие ключи усекаются до минимального различающего префикса. Реализовано в PostgreSQL **с версии 12**: при сплите листа новый разделитель (pivot tuple) сохраняет только те атрибуты, что нужны для разделения диапазонов.
- **Дедупликация (PG 13+)**: для неуникальных индексов ключ хранится один раз со списком TID.
- **Fill-фактор**: процент заполнения листовой страницы **при построении индекса** (по умолчанию в PostgreSQL 90%). Для активно пополняемых таблиц снижают до 70–80. Не путать со **средней заполненностью в устоявшемся состоянии**: при случайных вставках она стремится к ln 2 ≈ 69% независимо от начального fillfactor — это два разных числа, и их часто смешивают.

- **Bulk Insert**: при создании индекса по заполненной таблице данные сортируются, формируются листья, и дерево строится снизу вверх.
- **Pointer Swizzling**: замена ID страницы на прямой указатель после первого обращения (экономия на buffer manager). В PostgreSQL не реализован.

> **Дополнение из CMU 15-445/645 «Building B+Tree»**
> Самый быстрый и лучший способ построить B+-дерево — сначала отсортировать ключи, а затем построить индекс **снизу вверх**. Это объясняет, почему создание индекса на уже заполненной таблице (`CREATE INDEX`) значительно быстрее постраничной вставки.

---

## 5. Физическая упаковка страницы индекса (10 мин)

До сих пор мы рассуждали о дереве логически — узлы, ключи, указатели. Теперь спустимся на уровень ниже и посмотрим, как всё это уложено в байты на реальной странице PostgreSQL. Здесь нам наконец встретятся в «железе» те самые high key и rightlink, которые мы упомянули в разделе 2.1, рассказывая о Lehman–Yao.

Каждый узел B+-дерева — страница фиксированного размера (8 КБ). Внутри страница организована как отсортированный массив с косвенной адресацией, что позволяет быстрый бинарный поиск.

Структура страницы в PostgreSQL:

- **Заголовок (PageHeaderData)** — 24 байта: LSN страницы, checksum, флаги, границы областей (`pd_lower`, `pd_upper`, `pd_special`), размер страницы и версия формата, `pd_prune_xid`. Обратите внимание: **номера страницы в заголовке нет** (страница адресуется извне — файловым расположением), и **счётчика элементов нет** (число line pointers выводится из `pd_lower`).
- **Массив указателей (ItemIdData)** — каждый по 4 байта (смещение, флаги, длина), растёт от начала страницы. Именно этот массив **отсортирован по ключу** — логический порядок живёт здесь.
- **Свободное пространство** — между массивом указателей и областью данных.
- **Область данных** — IndexTuple (ключ + TID или указатель на дочернюю страницу), растёт от конца страницы к началу. Физический порядок кортежей произвольный.
- **Специальная область (BTPageOpaqueData)** в конце страницы:
  - `btpo_prev`, `btpo_next` — ссылки на **обоих** соседей по уровню (двусвязный список; rightlink — основа протокола Lehman–Yao);
  - `btpo_level` — уровень в дереве (0 — лист; внутренность страницы определяется по уровню > 0, отдельного флага «internal» нет);
  - `btpo_flags` — BTP_LEAF, BTP_ROOT, BTP_DELETED, BTP_META, BTP_HALF_DEAD, BTP_SPLIT_END, BTP_HAS_GARBAGE, BTP_INCOMPLETE_SPLIT;
  - `btpo_cycleid` — защита от повторного использования страниц (VACUUM cycle ID).

Ключевой момент: логический порядок ключей обеспечивается не физическим расположением IndexTuple, а сортировкой массива ItemId. Бинарный поиск выполняется по ключам, на которые указывают ItemId. При вставке новый кортеж кладётся в свободное место в области данных, а массив ItemId переупорядочивается.

**High key.** Первым элементом на любой странице, у которой есть правый сосед (и на листьях, и на внутренних), хранится **high key** — верхняя граница ключей этой страницы: он больше или равен

любому ключу на странице (и в её поддереве) и по смыслу равен минимальному ключу правого соседа. Пара (high key, rightlink) и есть механизм Lehman–Yao: если искомый ключ больше high key, значит, во время спуска произошёл конкурентный сплит, и нужно перейти вправо. На крайней правой странице уровня high key усечён до «плюс бесконечности».

Таким образом, B+-дерево физически — это набор страниц, каждая из которых внутри работает как отсортированный массив (через line pointers), обеспечивая быстрый бинарный поиск при эффективных вставках и удалениях.

> **Дополнение из Stanford CS245 «Node Size»**
> Размер узлов: n+1 указателей (фиксированный), n ключей. Минимальная заполненность для нелистовых узлов: ⌈(n+1)/2⌉ указателей, для листовых: ⌊(n+1)/2⌋ указателей на данные.

---

# 6. Где B+-дерево не работает: обзор родственных структур (20 мин)

Разделы 2–5 показали, насколько B+-дерево хорошо в своей задаче — точном и диапазонном поиске по упорядоченному ключу на диске. Теперь честно посмотрим на задачи, где оно **принципиально не работает**:

- проверка принадлежности «ключа точно нет», когда хранить сами ключи негде или незачем (→ фильтр Блума; вспомним различие «индекс vs фильтр» из раздела 1);
- конкурентный in-memory индекс, где нет ни страниц, ни сплитов (→ skip-список);
- строковые ключи, где важны префиксы, а не порядок значений (→ trie);
- вопрос не «где находится ключ?», а «в каких документах встречается термин?» (→ инвертированный индекс);
- поиск не по равенству, а по близости в векторном пространстве (→ векторные ANN-индексы).

Пройдёмся по ним в порядке «от почти индекса к совсем не индексу» — от точного ключа на диске до приближённого семантического поиска. Для каждой структуры разберём устройство, сложность и место в реальных системах.

## 6.1. Фильтр Блума (Bloom filter)

**Назначение**

Фильтр Блума — это **вероятностная структура данных** (битовая карта), отвечающая на запросы принадлежности множеству. Он даёт однозначный ответ «ключа точно нет» или «ключ, возможно, есть». **Ложноотрицательные результаты никогда не возникают, ложноположительные — могут.**

Это принципиальное отличие от индекса: фильтр не сообщает, *где* находится ключ, а только отвечает на вопрос, *существует* ли он.

**Устройство**

Фильтр Блума представляет собой битовый массив размера $m$ и $k$ хеш-функций.

- **Insert(x)**: каждая из $k$ хеш-функций вычисляет значение от ключа $x$, результат берётся по модулю $m$, и соответствующие биты в массиве устанавливаются в 1.

- **Lookup(x)**: вычисляются $k$ позиций; если хотя бы один бит равен 0 — ключа **точно нет**; если все биты равны 1 — ключ **возможно есть**.

Вероятность ложноположительного срабатывания зависит от размера битового массива, количества хеш-функций и числа вставленных элементов.

**Варианты**

- **Counting Bloom filter** — вместо битов используется массив целых чисел; поддерживает удаление ключей.
- **Cuckoo filter** — хранит не полные ключи, а их «отпечатки» (fingerprints); также поддерживает удаление.
- **Succinct range filter (SuRF)** — неизменяемый фильтр, позволяющий проверять наличие ключа в диапазоне.

**Алгоритмическая сложность**

| Операция | Лучший | Средний | Худший |
|---|---|---|---|
| Вставка | O($k$) | O($k$) | O($k$) |
| Поиск | O(1) — первый же проверенный бит равен 0 | O($k$) | O($k$) |
| Память | O($m$) бит | | |

где $k$ — число хеш-функций (обычно 3–10), $m$ — размер битового массива. Ранний выход при нулевом бите — типичная оптимизация.

**Применение в СУБД и бизнес-задачах**

Фильтры Блума широко используются в системах с большими объёмами данных для предотвращения дорогих операций чтения:

- **Cassandra и HBase** — фильтры Блума на уровне SSTable позволяют пропускать файлы, гарантированно не содержащие искомый ключ, отсекая значительную часть обращений к диску.
- **PostgreSQL** — расширение `bloom` позволяет создавать индекс-фильтр для ускорения поиска по нескольким колонкам.
- **Google Chrome** — фильтры Блума использовались в механизме Safe Browsing для локальной проверки URL.
- **Bitcoin SPV-клиенты (BIP-37)** — фильтры Блума позволяют запрашивать только релевантные транзакции.
- **LSM-деревья (RocksDB, LevelDB)** — фильтры на каждом файле данных ускоряют поиск ключей.

**Бизнес-кейс:** в рекомендательных системах (например, Medium) фильтры Блума используются для дедупликации — быстрой проверки, не рекомендовали ли мы этот материал пользователю ранее.

## 6.2. Skip-список (Skip list)

**Назначение**

Skip-список — это **вероятностная многоуровневая структура данных** из связных списков, предназначенная для быстрого поиска, вставки и удаления в отсортированном множестве. Основное преимущество — простота реализации (в том числе lock-free) при средней сложности, сравнимой со сбалансированными деревьями.

**Устройство**

Skip-список состоит из нескольких уровней связных списков:

- **Нижний уровень (Level 0)** — полный отсортированный список всех элементов.
- **Каждый следующий уровень** содержит подмножество элементов предыдущего уровня, позволяя «перепрыгивать» через группы узлов.
- Уровень нового узла определяется случайно (подбрасыванием монетки): с вероятностью 1/2 узел поднимается на следующий уровень. В среднем элемент имеет ~2 указателя, отсюда линейная ожидаемая память.

**Поиск:** начинается с верхнего уровня; движение вперёд, пока следующий узел не превысит искомое значение, затем спуск на уровень ниже.

**Вставка:** монетка определяет уровень узла; вставка выполняется **снизу вверх** на всех уровнях, чтобы читатели из других потоков всегда видели целостную структуру.

**Удаление:** часто реализуется лениво (пометка deleted) с фоновой очисткой.

**Алгоритмическая сложность**

| Операция | Лучший | Средняя | Худшая |
|---|---|---|---|
| Поиск | O(1) — элемент найден сразу на верхних уровнях | O(log *n*) | O(*n*) — вырожденное распределение уровней |
| Вставка | O(1) после нахождения позиции | O(log *n*) | O(*n*) |
| Удаление | O(1) после нахождения | O(log *n*) | O(*n*) |
| Память | O(*n*) | **O(*n*)** (в среднем ~2 указателя на элемент) | O($n \cdot L_{max}$) — при ограничении максимального уровня |

Средняя высота списка — O(log *n*), что обеспечивает логарифмическое среднее время поиска. Худший случай требует специально подобранного (или катастрофически неудачного) распределения «монеток» и на практике исчезающе маловероятен.

**Применение в СУБД и бизнес-задачах**

Skip-списки оптимальны для **in-memory** сценариев, где важна простота и конкурентность:

- **Memtable в RocksDB, LevelDB, Cassandra** — skip-список (в Cassandra — `ConcurrentSkipListMap`) используется как основная in-memory структура для сортировки данных перед записью в SSTable.
- **SingleStore (MemSQL)** — skip list — тип индекса по умолчанию в in-memory rowstore.

- **Redis (Sorted Sets)** — внутренняя реализация упорядоченных множеств использует skip-список (в сочетании с хеш-таблицей).
- **Memgraph (графовая БД)** — skip-списки используются для реализации индексов и ограничений уникальности.

**Бизнес-кейс:** в системах реального времени (обработка событий, IoT) skip-списки в memtable позволяют принимать миллионы вставок в секунду, группируя их для пакетной записи на диск.

## 6.3. Префиксное дерево (Trie / Radix tree)

**Назначение**

**Trie** (префиксное дерево, цифровое дерево) — структура, хранящая ключи посимвольно (побитово). **Radix tree** (сжатое префиксное дерево) — оптимизированная версия, объединяющая однонаправленные узлы для экономии памяти.

Ключевое преимущество — сложность поиска зависит от **длины ключа**, а не от количества элементов в структуре.

**Устройство**

- В trie каждый узел соответствует одному символу (или биту) ключа; путь от корня к листу представляет полный ключ.
- **Radix tree** сжимает узлы с одним потомком, уменьшая высоту дерева.
- **Adaptive Radix Tree (ART**, Leis, Kemper, Neumann, 2013) — современная реализация, адаптивно меняющая тип узлов (Node4, Node16, Node48, Node256) в зависимости от количества потомков для оптимального использования памяти. В статье доказана верхняя граница потребления — **не более 52 байт на ключ даже для произвольно длинных ключей**; на практике расход заметно ниже — например, около **8 байт на ключ** для плотных целочисленных ключей. Не путайте: 52 байта — гарантированный худший случай, а не «среднее».

**Сложность поиска** — $O(k)$, где $k$ — длина ключа в битах или символах.

**Алгоритмическая сложность**

| Операция | Лучший | Средний | Худший |
|---|---|---|---|
| Поиск | $O(k)$ | $O(k)$ | $O(k)$ |
| Вставка | $O(k)$ | $O(k)$ | $O(k)$ |
| Память | Сильно пересекающиеся префиксы: O(длина общих префиксов + уникальные суффиксы) | $O(\Sigma\|key_i\|)$, сжатая radix-версией | $O(\Sigma\|key_i\|)$ без общих префиксов |

Сложность не зависит от числа ключей $n$ — только от длины ключа. Это делает trie привлекательным для очень больших множеств коротких ключей.

**Применение в СУБД и бизнес-задачах**

Долгое время trie редко применялись в дисковых СУБД из-за плохой производительности при изменении на диске. Однако с ростом in-memory и LSM-архитектур ситуация меняется:

- **Apache Cassandra 5.0** — trie-структуры применены в memtable (TrieMemtable) и в новом trie-индексированном формате SSTable (BTI), дав заметный выигрыш по памяти и производительности.
- **DuckDB** — использует ART для первичных и вторичных индексов; ART исторически вырос из исследований HyPer (TUM), ставшего ядром Tableau.
- **Автодополнение (autocomplete)** — классический кейс: поиск всех слов с заданным префиксом.
- **Маршрутизация IP-адресов** — radix tree используется в сетевых таблицах маршрутизации (longest prefix match).

**Бизнес-кейс:** в поисковых системах и e-commerce trie-индексы позволяют мгновенно предлагать подсказки при вводе запроса (автодополнение) без сканирования всего каталога.

## 6.4. Инвертированный индекс (Inverted Index)

### Назначение

Инвертированный индекс — структура для **полнотекстового поиска**. В отличие от B+-дерева, которое отвечает на вопрос «где находится ключ?», инвертированный индекс отвечает на вопрос «в каких документах встречается термин?».

### Устройство

Инвертированный индекс состоит из двух частей:

1. **Словарь (Dictionary)** — все уникальные термины (слова). Обычно хранится в B-дереве или хеш-таблице.
2. **Постинг-листы (Posting lists)** — для каждого термина список идентификаторов документов, где этот термин встречается, часто с дополнительной информацией (частота, позиции).

**Поиск:** термин ищется в словаре; извлекается постинг-лист; выполняется операция над списками (пересечение, объединение) для составных запросов.

### Алгоритмическая сложность

| Операция | Лучший | Средний | Худший |
|---|---|---|---|
| Поиск термина в словаре | O(1) — хеш-таблица | O(log $T$) — B-tree по $T$ терминам | O($T$) — коллизии/вырождение хеша |
| Извлечение постинг-листа | O(1) после нахождения термина | O($L$) на чтение $L$ постингов | O($L$) |
| Пересечение списков | O(min($L_1$, $L_2$)) | O($L_1$ + $L_2$) | O($L_1$ + $L_2$) |
| Память | — | O($P$), где $P$ — суммарное число постингов | O($T$ × $D$) — вырожденный случай, когда каждый термин |

| Операция | Лучший | Средний | Худший |
|---|---|---|---|
| | | (вхождений терминов) | есть в каждом документе |

Поиск без индекса (полное сканирование) — O($D \cdot M$), где $D$ — число документов, $M$ — средняя длина. Инвертированный индекс сокращает это до чтения только релевантных постинг-листов — на больших коллекциях ускорение составляет порядки величины.

**Применение в СУБД и бизнес-задачах**

- **PostgreSQL GIN (Generalized Inverted Index)** — реализация инвертированного индекса для полнотекстового поиска по типам `tsvector`/`tsquery`. Недостаток GIN — медленное обновление: вставка одной строки может вызвать множество вставок в индекс. Начиная с PostgreSQL 8.4, GIN поддерживает отложенное обновление (fastupdate) через очередь ожидающих записей (pending list).
- **RUM-индекс** — расширение GIN, хранящее дополнительную позиционную информацию для ускорения ранжирования.
- **Elasticsearch / Lucene** — инвертированные индексы — основа полнотекстового поиска.
- **Системы поиска по логам** (например, стек Elastic: Logstash/Beats + Elasticsearch) — инвертированные индексы поверх распределённого хранилища позволяют искать по миллиардам записей журналов за доли секунды.

**Бизнес-кейс:** поиск по документам в корпоративных системах (библиотеки, CRM, базы знаний). Инвертированный индекс позволяет находить документы по ключевым словам за миллисекунды вместо минут полного сканирования.

## 6.5. Векторные индексы (Vector Indexes)

**Назначение**

Векторные индексы предназначены для **поиска по семантической близости** (similarity search) в многомерном пространстве — например, поиск похожих изображений, текстов, эмбеддингов. В отличие от точных индексов, векторные индексы обычно являются **приближёнными** (ANN — Approximate Nearest Neighbor), жертвуя точностью ради скорости.

**Основные алгоритмы**

**IVFFlat (Inverted File Flat):**

- Данные разбиваются на *списки* (кластеры) с помощью алгоритма кластеризации.
- При поиске проверяются только ближайшие списки (параметр `probes`).
- Рекомендуется строить после загрузки данных: построенный на пустой таблице индекс будет иметь нерепрезентативные центроиды и низкую точность.
- Быстрое построение, низкое потребление памяти, но может давать более низкую точность.

**HNSW (Hierarchical Navigable Small World):**

- Многослойный граф, где каждый слой — «маленький мир» с быстрым доступом (конструкция родственна skip-списку: экспоненциально редеющие слои).
- Поиск начинается с верхнего слоя и спускается вниз.

- Можно строить на пустой таблице (инкрементально).
- Более высокая точность (recall) и скорость запросов, но требует больше памяти и времени на построение.

**Алгоритмическая сложность**

| Алгоритм | Операция | Лучший | Средний | Худший |
|---|---|---|---|---|
| Точный (полный перебор) | Поиск | $O(n \cdot d)$ | $O(n \cdot d)$ | $O(n \cdot d)$ |
| IVFFlat | Построение | $O(n \cdot d \cdot clusters)$ | $O(n \cdot d \cdot clusters)$ | $O(n \cdot d \cdot clusters \cdot iters)$ |
| IVFFlat | Поиск | $O(clusters \cdot d + L_{min} \cdot d)$ — крошечные списки | $O(probes \cdot n/clusters \cdot d)$ | $O(n \cdot d)$ при $probes = clusters$ |
| HNSW | Построение | $O(n \cdot \log n \cdot d)$ | $O(n \cdot \log n \cdot d)$ | $O(n^2 \cdot d)$ — вырожденный граф |
| HNSW | Поиск | $O(d)$ — сосед найден на первом шаге | $O(\log n \cdot d)$ | $O(n \cdot d)$ |
| HNSW | Память | — | $O(n \cdot (M + d))$, где $M$ — средняя степень связей (константа) | — |

где $n$ — число векторов, $d$ — размерность, $clusters$ — число кластеров, $probes$ — число проверяемых кластеров, $L_{min}$ — размер наименьшего проверенного списка. Ожидаемая высота узла в HNSW — константа (геометрическое распределение, как в skip-списке), поэтому память растёт линейно по $n$, а не как $O(n \log n)$.

**Применение в СУБД и бизнес-задачах**

- **pgvector (PostgreSQL)** — расширение, поддерживающее IVFFlat и HNSW. HNSW рекомендуется для большинства production-нагрузок благодаря лучшему соотношению скорости и точности.
- **Weaviate, Milvus, Pinecone** — специализированные векторные БД.
- **RAG (Retrieval-Augmented Generation)** — поиск релевантных документов по эмбеддингам для LLM.
- **Рекомендательные системы** — поиск похожих товаров/контента.
- **Поиск изображений и видео** — семантический поиск по визуальным эмбеддингам.

**Бизнес-кейс:** в e-commerce векторные индексы позволяют показывать «похожие товары» на основе эмбеддингов изображений или текстовых описаний, обрабатывая миллионы товаров за миллисекунды.

**Сводная таблица структур**

| Структура | Тип | Основное применение | Поиск (лучший / средний / худший) | Память |
|---|---|---|---|---|
| **B+-дерево** | Точный индекс | Универсальный дисковый индекс | O(log_m *N*) во всех случаях | O(*N*) |
| **Bloom filter** | Вероятностный фильтр | Проверка отсутствия ключа | O(1) / O(*k*) / O(*k*) | O(*m*) бит |
| **Skip list** | Вероятностный список | In-memory индекс, memtable | O(1) / O(log *n*) / O(*n*) | средняя O(*n*) |
| **Trie / Radix tree** | Точный префиксный | Строковые ключи, автодополнение | O(*k*) во всех случаях (*k* — длина ключа) | O(Σ\|key$_i$\|), ART ≤ 52 Б/ключ |
| **Inverted Index** | Инвертированный | Полнотекстовый поиск | O(log *T*) словарь + O(*L*) постинги | O(*P*) постингов |
| **IVFFlat** | Приближённый ANN | Векторный поиск (быстрое построение) | — / O(*probes* · *n/clusters* · *d*) / O(*n* · *d*) | O(*n* · *d*) |
| **HNSW** | Приближённый ANN | Векторный поиск (высокая точность) | O(*d*) / O(log *n* · *d*) / O(*n* · *d*) | O(*n* · (*M* + *d*)) |

## 7. Итоги (5 мин)

- B+-дерево — основной индекс благодаря сбалансированности, высокому fanout и поддержке диапазонных запросов.
- Составные индексы и префиксный поиск дают гибкость, недоступную хеш-индексам.
- Дубликаты разрешаются добавлением TID к ключу; NULL-значения управляются NULLS FIRST/LAST.
- Физически страница индекса — массив с косвенной адресацией (ItemId); пара high key + rightlink из алгоритма Lehman–Yao (B-link tree) обеспечивает эффективное параллельное сканирование.
- На практике PostgreSQL реализует высококонкурентный вариант Lehman & Yao (nbtree), где полупустые страницы не сливаются, а полностью пустые удаляются VACUUM'ом (автоматически с PG 14); глубокий разбор латчинга — в следующей лекции.
- Там, где B+-дерево принципиально не работает — проверка принадлежности, полнотекстовый поиск, префиксные строки, семантическая близость, — применяются специализированные структуры: фильтры Блума, skip-списки, trie, инвертированные и векторные индексы. Как отмечает Энди Павло из CMU, именно комбинация этих структур позволяет современным СУБД закрывать широкий спектр задач — от точечных OLTP-запросов до аналитики и поиска по естественному языку.
- При всём этом B+-дерево остаётся «самой важной структурой данных в мире баз данных» (Andy Pavlo).

> **Заключительное дополнение из CMU 15-445/645**
> Почти каждая современная СУБД, поддерживающая индексы с сохранением порядка, использует

> B+-дерево. Оно спроектировано для дисковых СУБД и минимизирует *число* обращений к диску: при высоте дерева 3–4 точечный поиск стоит считанные единицы страниц. (Превращение случайной записи в последовательную — это уже территория LSM-деревьев, а не B+-деревьев.)
>
> **Из Stanford CS245**
> B+-деревья являются доминирующим методом индексации в реляционных СУБД благодаря сбалансированности, высокому коэффициенту ветвления и поддержке диапазонных запросов. Они успешно применяются для первичных и вторичных индексов, кластеризованных и некластеризованных индексов.

---

**Дополнительная литература:**


- PostgreSQL Documentation, Chapters 11, 61.
- Исходный код PostgreSQL: `src/backend/access/nbtree/README` (лучший первоисточник по high key, сплитам и удалению страниц).
- Bayer R., McCreight E. «Organization and Maintenance of Large Ordered Indices» (Acta Informatica, 1972).
- Comer D. «The Ubiquitous B-Tree» (ACM Computing Surveys, 1979).
- Lehman P., Yao S.B. «Efficient Locking for Concurrent Operations on B-Trees» (ACM TODS, 1981).
- Leis V., Kemper A., Neumann T. «The Adaptive Radix Tree» (ICDE, 2013).
- Gray, Reuter. «Transaction Processing: Concepts and Techniques».
- CMU 15-445/645 Lecture Notes on Indexes.
- Stanford CS245/CS346 Course Materials.